Aulão AEDS II

POO, Estruturas de Dados e Ordenação

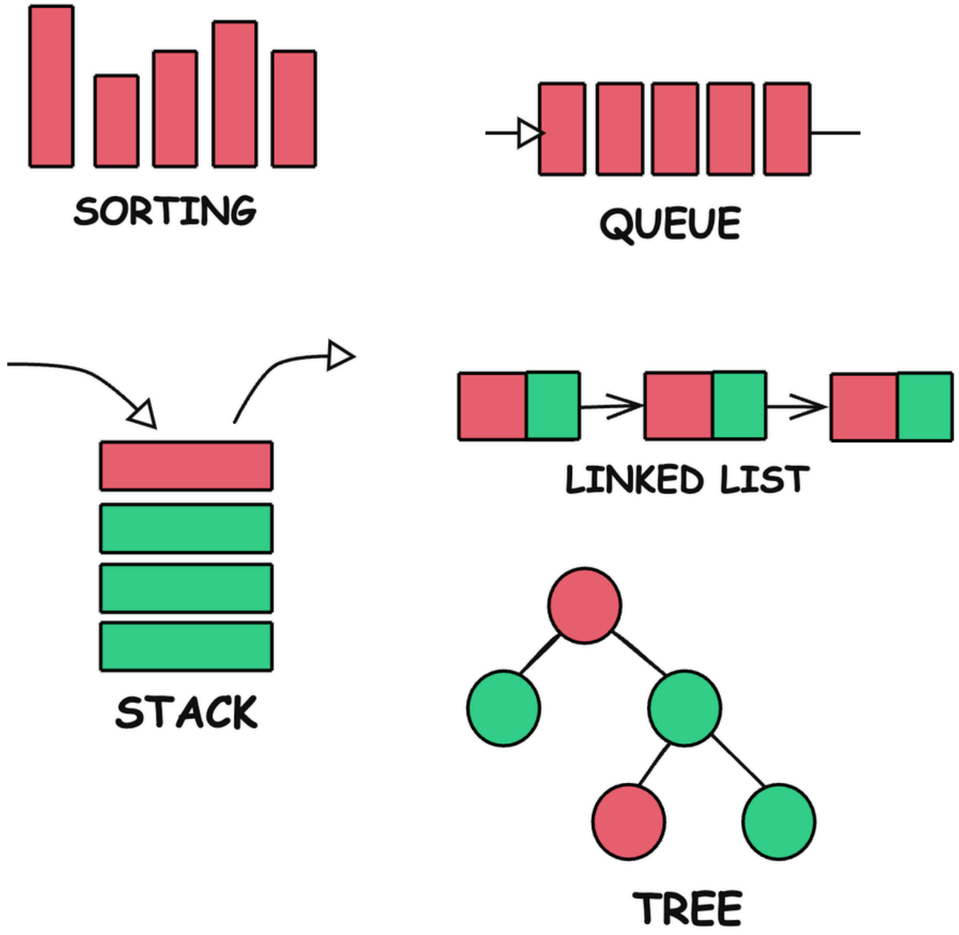
Daniel Gomes Pereira
Pedro Italo Borges Cardoso

O que será visto

1. Introdução
2. Ambiente (vim + compilação)
3. Revisão rápida de conceitos (Ponteiros, Malloc, Struct)
4. POO com Java
5. Estruturas de dados (Pilha, Fila, Lista)
6. Algoritmos de ordenação
7. Problema integrador

1. Introdução

- Como organizar dados de forma eficiente em um programa?
- Qual estrutura usar para cada tipo de problema?
- A resposta está nas estruturas de dados e nos algoritmos.



2. Ambiente (vim + compilação)

- Criação e salvamento de arquivos
- Modos do vim
- Atalhos básicos

2.1 Criação e salvamento de arquivos

- **Criar um arquivo**
  vim nome.extensão

2.1.1 Criação e salvamento de arquivos

- Fechar o arquivo
  - Salvando

    :w
  - Salvar e sair

    :wq
  - Não salvando
    - Forçado

      :q!
  - Não forçado

    :q

2.2 Modos do vim

- **Editar (botão Insert ou i)**
- **Visualização (v)**
- **Normal (vem padrão ou apertar esc enquanto no outro modo)**

2.3 Atalhos básicos

Obs: Para utilizar os atalhos, é preciso estar no modo normal (esc)

- **Copiar e deletar a linha (dd)**
  - **copiar e deletar varias linhas (d<quantidade de linhas> <direção>)**
- **Copiar sem deletar a linha(yy)**
  - **Copiar sem deletar varias linhas(y<quantidade de linhas><direção>)**

2.3.1 Atalhos básicos

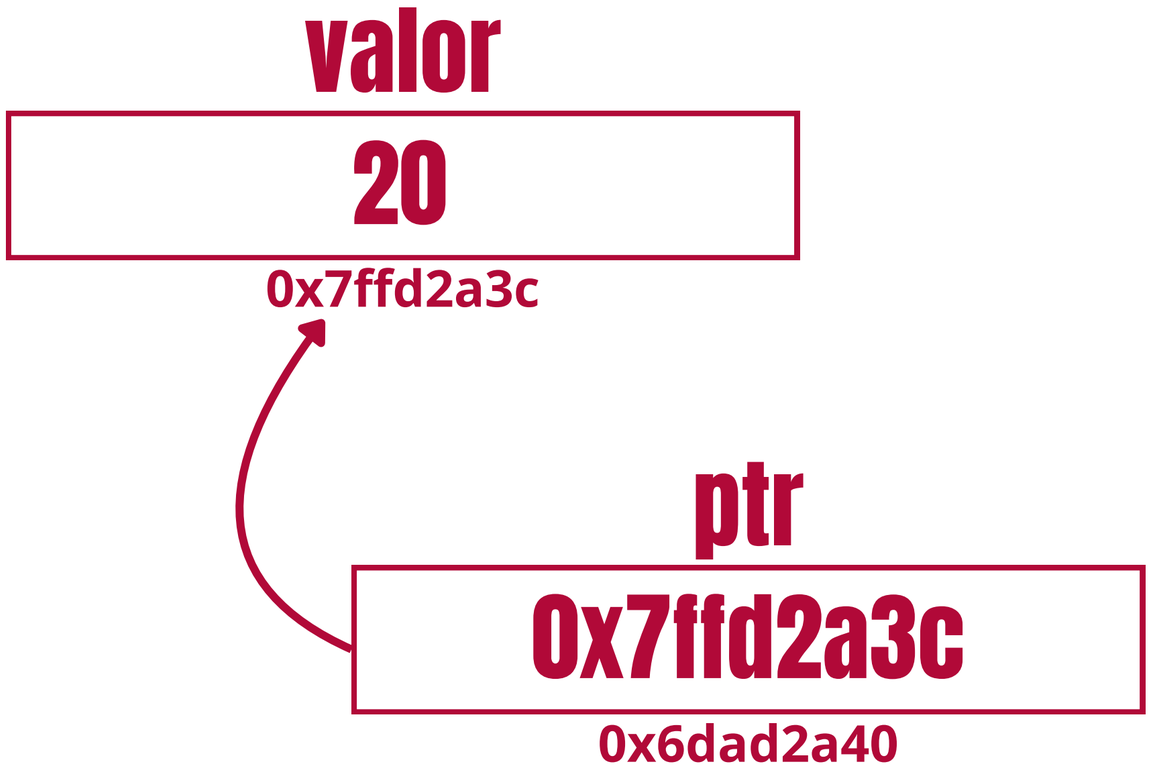
Obs: Para utilizar os atalhos, é preciso estar no modo normal (esc)

- **Colar embaixo da linha atual(p)**
- **Colar em cima da linha atual(shift + p)**
- **Reverter erros(u)**
- **Reverter o reverter(ctrl + r)**
- **Avançar para a proxima palavra(w)**
- **Voltar para a palavra anterior(b)**
- **Ir para o fim/ inicio da página(end/home)**

3. Revisão rápida de conceitos

- **Ponteiros**
- **Malloc**
- **Struct**

3.1 Ponteiros



- **Ponteiros**
  - **Variáveis que armazenam endereços de memória**
  - **Permitem manipular dados diretamente na memória**
  - **Muito usados em estruturas de dados dinâmicas**

3.1 Ponteiros

```c
int main() {
    int valor = 0;
    int *ptr = &valor;
    printf("%p %p %d %d", &valor, ptr, *ptr, valor);

    // Saida
    // 0x7ffe6d2d922c 0x7ffe6d2d922c 0 0
    // ---------------------------------------------------
    // 0x7ffe6d2d922c  | 0x7ffe6d2d922c | 0     | 0
    // &valor          | ptr            | *ptr  | valor
}
```

3.2 Malloc

- **Malloc**
  - **Reserva memória em tempo de execução**
  - **Retorna o endereço da memória alocada**
  - **Muito usado em estruturas de dados dinâmicas**

3.2 Malloc

- **Malloc**
  - **A função malloc retorna um ponteiro do tipo void*.**
  - **Por isso, normalmente fazemos casting, ou seja, convertemos o ponteiro para o tipo desejado.**

3.2 Malloc

```c
#include <stdlib.h>

int main() {
    int size = 6;
    int *vetor = (int*) malloc(sizeof(int) * size);

    return 0;
}
```

int - 4 Bytes - 32 bits

3.3 Struct

- **Struct**
  - **Permite agrupar diferentes tipos de dados em uma única estrutura**
  - **Cada campo ocupa uma posição na memória**

3.3 Struct

```c
#include <stdlib.h>

struct Pessoa {
    char* Nome;
    int idade;
};

int main() {
    struct Pessoa novaPessoa1; // Criado na STACK

    struct Pessoa *novaPessoa2 = (struct Pessoa*) malloc(sizeof(struct Pessoa)); // Criado na HEAP

    free(novaPessoa2); // Necessita de free
    return 0;
}
```

3.3 Struct

- **Utilizamos typedef para evitar reescrever a declaração completa da estrutura.**

- **O typedef cria um apelido para um tipo de dado.**

- **No caso das structs, ele permite usar apenas o nome da estrutura, sem precisar escrever struct a cada declaração.**

3.3 Struct

```c
#include <stdlib.h>

typedef struct Pessoa {
    char* Nome;
    int idade;
} Pessoa;

int main() {
    Pessoa novaPessoa1; // Criado na STACK

    Pessoa *novaPessoa2 = (Pessoa*) malloc(sizeof(Pessoa)); // Criado na HEAP

    free(novaPessoa2); // Necessita de free
    return 0;
}
```

3.3 Struct

- **Formas de acesso**
  - **->: O operador '->' é usado quando temos um ponteiro para a estrutura.**
  - **. : Usamos o operador '.' quando temos a estrutura diretamente.**

3.3 Struct

```c
typedef struct Pessoa {
    char* Nome;
    int idade;
} Pessoa;

int main() {
    Pessoa novaPessoa1;
    Pessoa *novaPessoa2 = (Pessoa*) malloc(sizeof(Pessoa));

    novaPessoa1.idade = 18; // Acesso direto
    novaPessoa2->idade = 20; // Acesso por meio de ponteiro

    free(novaPessoa2);
    return 0;
}
```

4. Programação Orientada a Objetos

- **Paradigma de programação que organiza o código em classes e objetos.**

- **Cada objeto representa uma entidade do mundo real, possuindo:**
  - **Dados (atributos)**
  - **Comportamentos (métodos)**

4.1 Classe e Objeto

- **Classe é um molde que define:**
  - **Atributos (dados)**
  - **Métodos (comportamentos)**
  - **Construtor (inicialização do objeto)**
- **Objeto é uma instância de uma classe, ou seja, algo criado a partir do molde.**

4.1 Classe e Objeto

```
class Pessoa {
    String nome;
    int idade;

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    void apresentar() {
        System.out.println("Nome: " + nome + ", Idade: " + idade);
    }
}
```

4.1 Classe e Objeto

- **Instanciar é o processo de criar um objeto a partir de uma class**

4.1 Classe e Objeto

```
class Programa {
    public static void main(String[] args) {
        Pessoa novaPessoa = new Pessoa("Ana", 18);

        nova.apresentar();
    }
}
```

4.1 Classe e Objeto

- new → cria o objeto na memória
- Pessoa(...) → chama o construtor
- novaPessoa → referência para o objeto criado

4.1 New e Malloc

- new (Java)
  - Aloca memória automaticamente
  - Já chama o construtor
  - Possui gerenciamento automático de memória (Garbage Collector)
- malloc (C)
  - Apenas aloca memória
  - Não inicializa valores
  - Gerenciamento manual (free)

4.2 Características

- **Encapsulamento**
- **Abstração**
- **Herança**
- **Polimorfismo**
- **Sobrecarga**

4.2.1 Encapsulamento

Agrupar dados (atributos) e comportamentos (métodos) em uma classe, ocultando detalhes internos e protegendo o estado do objeto.

4.2.1 Encapsulamento

Modificadores de acesso:

- **São palavras-chave que controlam quem pode acessar atributos e métodos de uma classe.**
  - **private: Só a própria classe acessa;**
  - **public: Qualquer classe pode acessar;**
  - **protected: A própria classe ou classes filhas (herança)**

4.2.1 Encapsulamento

```
class Pessoa {
    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

4.2.2 Abstração

É o processo de simplificar objetos do mundo real, focando apenas nas características e comportamentos essenciais para o software e ignorando detalhes complexos desnecessários.

4.2.2 Abstração

```
class Animal {
    protected String especie;
    protected String idade;

    public void comer() {
        System.out.println("Comendo");
    }
}
```

4.2.3 Herança

È um pilar que permite a uma subclasse (filha) herdar atributos e métodos de uma superclasse (base/pai)

4.2.3 Herança

```
class Animal {
    protected String especie;
    protected String idade;

    public void comer() {
        System.out.println("Comendo");
    }
}

class Gato extends Animal {
    void miar() {
        System.out.println("Miando");
    }
```

4.2.4 Polimorfismo

Permite que classes derivadas de uma base utilizem métodos com a mesma assinatura, mas com comportamentos específicos e diferentes para cada uma

4.2.4 Polimorfismo

```
class Animal {
    void emitirSom() {
        System.out.println("Som genérico");
    }
}

class Cachorro extends Animal {
    void emitirSom() {
        System.out.println("Latindo");
    }
}
```

4.2.5 Sobrecarga

Permite que uma classe tenha múltiplos métodos com o mesmo nome, desde que possuam assinaturas diferentes (quantidade ou tipo de parâmetros).

4.2.5 Sobrecarga

```
class Calculadora {

    int soma(int a, int b) {
        return a + b;
    }

    int soma(int a, int b, int c) {
        return a + b + c;
    }

    double soma(double a, double b) {
        return a + b;
    }
}
```

4.3 Estrutura básica em JAVA

A estrutura básica em Java define como um programa deve ser organizado para ser executado.

4.2 Estrutura básica em JAVA

Todo programa Java precisa ter:

- **Uma classe**
- **Um método principal (main)**
- **Código dentro de métodos**

4.2 Estrutura básica em JAVA

```
public class Programa {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

4.2 Estrutura básica em JAVA

- `public class Program` → **define a classe**
- `public static void main(String[] args)` → **método principal**
- `System.out.println()` → **saída de dados**

4.3 Entrada e Saída Padrão

- **Entrada → dados fornecidos pelo usuário**
- **Saída → dados exibidos pelo programa**

4.3 Saída Padrão

- **Utilizada para mostrar informações na tela:**
  - **System.out.println("Olá, mundo!");**
    - **System.out → saída padrão**
    - **println → imprime com quebra de linha**

4.3 Entrada Padrão

- **Utilizada para receber dados do usuário:**
  - **import java.util.Scanner;**
  - **Scanner sc = new Scanner(System.in);**
  - **String nome = sc.nextLine();**
    - **System.in → entrada padrão**
    - **Scanner → lê dados do teclado**

4.3 Entrada Padrão

```
import java.util.Scanner;
public class Programa {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Digite seu nome:");
        String nome = sc.nextLine();
        System.out.println("Olá, " + nome);
    }
}
```

4.3 MyIO

- A MyIO é uma classe utilitária que facilita a entrada e saída de dados em Java, substituindo o uso direto de:
  - System.out
  - Scanner
  - System.in

4.3 MyIO

- Centraliza entrada e saída em uma única classe
- Permite ler diferentes tipos de dados facilmente
- Controla o charset (codificação de caracteres)
- Evita repetição de código

4.3 MyIO - Exemplos

- **Saída de dados**
  - **MyIO.println("Olá, mundo!");**
  - **MyIO.print(10);**
  - **MyIO.println(3.14);**

4.3 MyIO - Exemplos

- **Entrada de dados**
  - **String nome = MyIO.readLine();**
  - **int idade = MyIO.readInt();**
  - **double altura = MyIO.readDouble();**
  - **String titulo = MyIO.readString();**

4.3 MyIO - Exemplos

```
class Programa {
    public static void main(String[] args) {

        String nome = MyIO.readLine("Digite seu nome: ");
        int idade = MyIO.readInt("Digite sua idade: ");
        double altura = MyIO.readDouble("Digite sua altura: ");

        MyIO.println("\n--- Dados informados ---");
        MyIO.println("Nome: " + nome);
        MyIO.println("Idade: " + idade);
        MyIO.println("Altura: " + altura);

        int anoNascimento = 2026 - idade;
        MyIO.println("Ano de nascimento aproximado: " + anoNascimento);
    }
}
```